

ĐINH LÊ THÀNH CÔNG

STT: 14

Pentest Android

Người hướng dẫn: **Nguyễn Lê Minh, Lê Quang Minh**

Mục lục

1	Kiến trúc Android	3
1.1	Cấu trúc file APK	3
1.2	Công cụ	3
2	Kỹ thuật thường gặp	3
2.1	Root detection	3
2.2	Hooking	3
2.3	Patching	3
2.4	Detect Frida	4
2.5	DEX encrypt & dynamic loading	4
2.6	SSL Pinning	4
2.7	Native Library & JNI Binding	4
3	App thực hiện	5
3.1	HTB Infinity Bank	5
3.2	V-app	5

1. Kiến trúc Android

- Android là một kiến trúc dạng nhiều tầng, được xây dựng trên nền Linux kernel. Mỗi tầng cung cấp dịch vụ cho tầng phía trên, giúp ứng dụng có thể tương tác với phần cứng thông qua các API chuẩn. Các thành phần chính bao gồm: Linux Kernel, HAL, Android Runtime, Native lib, Java API Framework, System Apps.

1.1 Cấu trúc file APK

- APK là gói cài đặt của ứng dụng Android, bản chất là một file nén chứa thông tin cần thiết để cài đặt và chạy ứng dụng. Đối với ứng dụng Android native viết bằng Java/Kotlin, cấu trúc APK thường gồm các thành phần chính: AndroidManifest.xml, classes.dex, res, resources.arsc, assets, lib, META-INF.
- Khi APK decompile, mã trong classes.dex sẽ được chuyển sang dạng smali. Đây là mã assembly-level của Dalvik bytecode.

1.2 Công cụ

- Một số công cụ phổ biến như: ADB, JADX, Apktool, Frida, IDA Pro, Magisk, KSU, Apksigner/Jarsigner, MobSF, ...

2. Kỹ thuật thường gặp

2.1 Root detection

- Kỹ thuật kiểm tra thiết bị có dấu hiệu bị root hoặc bị can thiệp môi trường như unlock bootloader, custom ROM, Magisk/Zygisk. Cơ chế này thường được đặt ở nhiều vị trí trong app và kết hợp với anti-tampering.
- Các cơ chế detection: Basic checks, File checks, Package checks, Process checks, Native checks, Server-side attestation.

2.2 Hooking

- Kỹ thuật can thiệp vào hàm/method tại runtime nhằm quan sát hoặc thay đổi hành vi chương trình, ví dụ log tham số, log giá trị trả về, thay đổi return value, gọi lại hàm gốc hoặc thay thế implementation.
- Hooking detection: kỹ thuật phát hiện app đang bị can thiệp runtime bởi các công cụ như Frida, Xposed/LSPosed, Substrate, debugger hoặc các kỹ thuật hook native. Một số cách phát hiện gồm: Kiểm tra process/module lạ trong mem; phát hiện Frida server/agent; kiểm tra debugger bằng TracerPid; phát hiện dấu hiệu Xposed trong ClassLoader/stack trace/package; kiểm tra inline hook hoặc GOT/PLT hook ở tầng native.

2.3 Patching

- Là việc APK bị chỉnh sửa so với bản gốc, ví dụ sửa bytecode, native .so, tài nguyên, file cấu hình, manifest hoặc ký lại APK.
- Patching detection: kỹ thuật phát hiện ứng dụng có bị sửa đổi hay không. Một số cách phổ biến gồm: kiểm tra certificate/signature của APK; kiểm tra nguồn cài, so sánh hash của classes, native lib; kiểm tra manifest bất thường permission/component bị thêm; kiểm tra integrity ở tầng native.

- Bài phân tích chi tiết về root, hook, patch: [LyThuyet](#).

2.4 Detect Frida

- **Checking integrity:** kỹ thuật phát hiện Frida bằng cách so sánh integrity của code trên disk và code đang được load trong memory. Khi Frida hook một hàm native, nó thường sửa một số instruction ở đầu hàm để chuyển hướng luồng thực thi sang trampoline/hook handler. Do đó, bytes của hàm hoặc vùng `.text` trong memory sẽ khác so với bản gốc trên disk. Cách kiểm tra phổ biến:
 - Đọc thông tin mapping của thư viện từ `/proc/self/maps`, chọn vùng có quyền thực thi như `r-xp`, sau đó tính CRC/hash của cùng một vùng code trên disk và trong memory. Nếu hai giá trị khác nhau, có thể nghi ngờ thư viện hoặc hàm đã bị hook/patch runtime.
 - Check một số lib bất thường trong `/proc/self/maps` như Frida-agent, ...
- **Detect Frida by stacktrace:** Kỹ thuật kiểm tra call stack của hàm nhảy cảm. Bình thường hàm native được gọi theo luồng Java/Kotlin → ART → JNI → native `.so`. Khi Frida hook vào native function, entry point của hàm có thể bị patch để nhảy sang trampoline/hook handler. Luồng gọi này không còn đúng với call chain ban đầu, khiến stack trace bị thiếu frame ART/JNI, bị truncate, xuất hiện anonymous executable memory hoặc module đáng ngờ. Từ đó app có thể phát hiện dấu hiệu runtime instrumentation.
- Bài viết chi tiết và video demo: [detect frida](#).

2.5 DEX encrypt & dynamic loading

- Kỹ thuật mã hóa file một file DEX/JAR/APK chứa `classes.dex` và lưu trong app, thường ở `assets`. Khi app chạy, file này được giải mã ra thư mục `private` của ứng dụng, sau đó nạp động bằng cơ chế như `DexClassLoader`. Cách này giúp che giấu một phần logic chương trình, làm quá trình phân tích tĩnh khó hơn, nhưng cần kiểm tra integrity vì file được giải mã hoặc nạp động có thể bị thay thế.
- Video demo: [Dex and dynamic](#).

2.6 SSL Pinning

- Kỹ thuật giúp app chỉ tin tưởng đúng certificate/public key của server thay vì chỉ dựa vào CA hệ thống, nhằm giảm nguy cơ MITM. Một số kiểu phổ biến gồm certificate pinning, public key pinning và hash pinning. Kỹ thuật này có thể triển khai bằng `Network Security Config`, `OkHttp CertificatePinner` hoặc custom logic trong Java/Kotlin/native code. Nhận biết bằng phân tích tĩnh với JADX/MobSF hoặc phân tích động qua proxy Burp.
- Bypass SSL pinning phụ thuộc vào cách app triển khai, thường liên quan đến patch logic kiểm tra, hook hàm pinning hoặc thay đổi cấu hình trust trong môi trường kiểm thử hợp pháp.

2.7 Native Library & JNI Binding

- Java/Kotlin có thể gọi xuống hàm C/C++ trong file native `.so` thông qua JNI. Khi app gọi `System.loadLibrary()` hoặc `dlopen()`, Android linker sẽ load thư viện vào memory, map các segment ELF, resolve dependency, relocation GOT/PLT và chạy constructor như `.init_array`. Nếu thư viện export `JNI_OnLoad`, ART/Dalvik sẽ gọi hàm này sau khi load thư viện.
- JNI có hai cách chính để nối Java native method với hàm C/C++ trong `.so`:

- JNI naming convention: hàm C/C++ được export theo tên chuẩn như `Java_package_Class_method`. Khi native method được gọi, ART/Dalvik sẽ lookup symbol tương ứng trong `.so`, có thể hiểu gần giống cơ chế `dlsym`. Cách này không bắt buộc phải có `JNI_OnLoad`.
 - RegisterNatives: thường được gọi trong `JNI_OnLoad`, developer tự tạo bảng mapping giữa tên method Java, signature và địa chỉ hàm C/C++. Cách này cho phép hàm native đặt tên tùy ý, không cần export theo convention JNI.
- Bài phân tích chi tiết: [OWASP Mobile Lv4](#).

3. App thực hiện

3.1 HTB ([Infinity Bank](#))

- Challenge viết bằng Flutter, nên logic chính không nằm nhiều ở Java/Kotlin mà được biên dịch AOT vào `libapp.so`; `libflutter.so` đóng vai trò Flutter engine. Vì vậy, thay vì phân tích chủ yếu bằng JADX như app Android native, tập trung vào native library và dùng Blutter để khôi phục một phần thông tin Dart từ `libapp.so`.
- App sử dụng SSL pinning nên bị lỗi handshake/certificate. Bypass SSL pinning bằng Frida, hook vào hàm kiểm tra SSL trong `libflutter.so` và sửa giá trị trả về để app chấp nhận kết nối, từ đó có thể quan sát request/response qua Burp Suite.
- Phần PIN/OTP được xử lý cục bộ. Sau khi tìm được luồng xử lý PIN và khôi phục symbol Dart, trong đó `user_pin` được lấy từ `shared_prefs`, sau đó qua hàm `reverseXORBase64`. Cơ chế chính là base64 decode dữ liệu rồi XOR với hằng số để khôi phục PIN.
- Sau bước OTP, app gửi request với cơ chế mã hóa riêng: tạo AES key/IV/salt, tạo JSON body, tính signature bằng SHA-256 trên plaintext JSON, sau đó mã hóa body bằng AES-CBC. Các tham số AES tiếp tục được mã hóa bằng RSA-OAEP thông qua `librsa_bridge.so`.
- Do không hook trực tiếp được các hàm trong `libapp.so` trên môi trường x86_64 vì khác offset so với tool, nên hook vào `librsa_bridge.so` tại luồng RSA-OAEP để lấy AES key/IV/salt trước khi bị mã hóa. Sau đó dùng các giá trị này để giải mã request trong Burp và phân tích dữ liệu giao tiếp với server.
- Chi tiết bài viết và video thực hiện: [HTB](#).

3.2 V-app

- V-app là app viết bằng Flutter. Do quá trình thực hiện trên emulator x86_64, các tool hầu hết hỗ trợ ARM, việc phân tích sâu `libapp.so`, lấy offset và khôi phục symbol còn bị hạn chế. Vì vậy, phần này tập trung trước vào Java layer, Flutter bridge và các cơ chế runtime detection.
- Ở Java layer, package `com.absolutions.security_plus` chứa nhiều cơ chế kiểm tra như root, emulator, ADB, debugger, Frida, Magisk, Xposed, custom ROM và system property bất thường. Sử dụng Frida để hook các hàm này thay đổi return value, fake trạng thái thiết bị an toàn và chỉnh sửa các map trả về cho Flutter như device info, security status và biometric/security state.
- App cũng tích hợp Talsec/freeRASP. Flutter/Dart gọi xuống native Android thông qua `MethodChannel` với channel `talsec.app/freerasp/methods`. Khi Flutter gọi method `start`, native layer sẽ khởi động scanner và đăng ký receiver để nhận threat event.

- Dùng Frida để trace Intent, PackageManager và stack trace nhằm xác định cơ chế report threat. Kết quả cho thấy app sử dụng Intent action `TALSEC_INFO`, trong đó extra `INFO_DATA` chứa loại threat như emulator hoặc root. Ngoài ra, app còn kiểm tra Hide My Applist thông qua `resolveContentProvider` và `setClassName`.
- Kết quả hiện tại là đã hiểu được một phần luồng bảo vệ của app, bao gồm Flutter → Method-Channel → native checker → Intent `TALSEC_INFO` → Flutter, đồng thời xác định được các điểm kiểm tra ở Java layer và cơ chế report threat. Tuy nhiên, việc bypass vẫn chưa hoàn chỉnh vì app còn các lớp kiểm tra khác, đặc biệt ở native layer và Flutter AOT/native library.
- Hướng phát triển tiếp theo là chuyển sang môi trường thiết bị ARM thật để phân tích `libapp.so` hiệu quả hơn, khôi phục symbol/offset bằng các công cụ phù hợp, tiếp tục phân tích native layer, kiểm tra luồng request sau khi bypass hoàn toàn các lớp detection, và hoàn thiện quá trình bắt request trong môi trường kiểm thử.
- Chi tiết phân tích: [V-app](#).